بسم الله الرحمن الرحیم



موضوع: تکلیف سری دوم

درس هوش مصنوعی مقدماتی

نام و نام خانوادگی: محمدمهدی آزادی

شماره دانشجویی: 40214623

نام استاد: استاد تشنه‌لب

پروژه درس هوش مصنوعی مقدماتی

1404-1405

فهرست

Table of Contents

مقدمه و لینک Git-hub.....1

سوال اول: محاسبه Fitness و انتخاب (Selection) 2

سوال دوم: عملگر (Crossover) ترکیب 5

سوال سوم: عملگر (Mutation) جهش 8

پیاده‌سازی الگوریتم ژنتیک برای انتخاب ویژگی در دیتاست iris 12

مقدمه و لینک Git-hub

در عصر حاضر، با گسترش روزافزون حجم داده‌ها در سیستم‌های هوشمند، پدیده‌ای تحت عنوان «نفرین ابعاد» (Curse of Dimensionality) به یکی از چالش‌های اساسی در حوزه یادگیری ماشین و داده‌کاوی تبدیل شده است. وجود ویژگی‌های زائد، نامرتبط یا نویزدار در مجموعه‌داده‌ها، نه‌تنها بار محاسباتی و زمان آموزش مدل‌ها را به شدت افزایش می‌دهد، بلکه با ایجاد پدیده بیش‌برازش (Overfitting)، دقت و کارایی سیستم را نیز تنزل می‌بخشد. در این راستا، «انتخاب ویژگی» (Feature Selection) به عنوان یک گام پیش‌پردازش حیاتی، با هدف یافتن بهینه‌ترین زیرمجموعه از ویژگی‌ها، نقش بسزایی در ساده‌سازی مدل‌ها و ارتقای قدرت تعمیم‌پذیری آن‌ها ایفا می‌کند.

از آنجا که جستجوی جامع در فضای تمامی ترکیب‌های ممکنِ ویژگی‌ها نیازمند زمان محاسباتی نمایی است، روش‌های جستجوی ابتکاری اهمیت ویژه‌ای می‌یابند. پروژه حاضر، به پیاده‌سازی و بررسی عملکرد **الگوریتم ژنتیک** (**Genetic Algorithm**) {که یکی از قدرتمندترین روش‌های بهینه‌سازی مبتنی بر تکامل طبیعی است} برای مسئله انتخاب ویژگی می‌پردازد. در این پیاده‌سازی، با استفاده از دیتاست کلاسیک Iris و یک طبقه‌بند K-NN، نشان خواهیم داد که چگونه عملگرهای ژنتیکی مانند تقاطع، جهش و انتخاب هدفمند می‌توانند با ایجاد تعادل میان دقتِ دسته‌بندی و سادگیِ مدل، بهینه‌ترین ویژگی‌ها را استخراج نمایند.

لینک مخزن Git-hub: (کلیک کنید.)

https://github.com/mmehd8392-bot/Machine-Learning-Course.git

سوال اول: محاسبه Fitness و انتخاب (Selection)

۱ سوال اول: محاسبه Fitness و انتخاب (Selection)

فرض کنید یک الگوریتم ژنتیک برای حل مسئله مینیمم‌سازی تابع زیر طراحی شده است:

$$f(x) = x^2 - 4x + 7 \quad , \quad x \in [0, 10]$$

جمعیتی شامل ۴ کروموزوم با مقادیر زیر داریم:

- $x_1 = 1$
- $x_2 = 2$
- $x_3 = 5$
- $x_4 = 8$

الف)

تابع fitness مناسب برای این مسئله مینیمم‌سازی را تعریف کنید. (توضیح دهید چرا نمی‌توان مستقیماً از $f(x)$ استفاده کرد)

چالش: الگوریتم‌های انتخاب استاندارد مانند چرخ رولت، برای مسائل **بیشینه‌سازی** (Maximization) طراحی شده‌اند. آن‌ها به کروموزوم‌هایی با مقدار Fitness بیشتر، شانس انتخاب بالاتری می‌دهند. اگر ما مستقیماً از $f(x)$ به عنوان Fitness استفاده کنیم، الگوریتم سعی می‌کند مقادیر بزرگتر $f(x)$ را انتخاب کند که دقیقاً برعکس هدف ماست.

راه حل: تبدیل تابع هزینه به تابع برازندگی (fitness) با معکوس کردن انجام می‌شود. برای مسئله مینیمم‌سازی، معمولاً از رابطه های زیر استفاده می‌شود:

$$Fitness:\ F(x) = \frac{1}{f(x) + \epsilon} \quad (*)$$

$$Fitness:\ F(x) = C_{max} - f(x) \quad (**)$$

این مقدار ϵ مقدار بسیار کوچکی (مثلا در حد 10^{-3}) است که برای جلوگیری از صفر شدن مخرج گذاشته می‌شود؛ C_{max} هم بیشترین مقدار ممکن برای $f(x)$ می‌باشد.

ما اینجا از روش (**) استفاده می کنیم. ابتدا مقادیر $f(x)$ را برای جمعیت محاسبه کرده و سپس C_{max} را تعیین می‌کنیم.

ب)

مقدار fitness هر کروموزوم را محاسبه کنید.

جمعیت ما شامل کروموزوم‌های $x_1 = 1, x_2 = 2, x_3 = 5, x_4 = 8$است.

1. **محاسبه مقدار تابع هدف ($f(x)$) برای هر کروموزوم:**

- $f(x_1) = f(1) = (1)^2 - 4(1) + 7 = 1 - 4 + 7 = 4$
- $f(x_2) = f(2) = (2)^2 - 4(2) + 7 = 4 - 8 + 7 = 3$
- $f(x_3) = f(5) = (5)^2 - 4(5) + 7 = 25 - 20 + 7 = 12$
- $f(x_4) = f(8) = (8)^2 - 4(8) + 7 = 64 - 32 + 7 = 39$

2. **محاسبه مقدار تابع برازش ($F(x)$):**

بیشترین مقدار $f(x)$ در جمعیت فعلی برابر با 39 است. ما C_{max} را کمی بزرگتر از این مقدار، مثلاً $C_{max} = 40$، در نظر می‌گیریم تا از صفر شدن Fitness جلوگیری شود.

- $F(x_1) = 40 - f(1) = 40 - 4 = 36$
- $F(x_2) = 40 - f(2) = 40 - 3 = 37$
- $F(x_3) = 40 - f(5) = 40 - 12 = 28$
- $F(x_4) = 40 - f(8) = 40 - 39 = 1$

ج)

با استفاده از روش Roulette Wheel Selection، احتمال انتخاب هر کروموزوم را به دست آورید.

1. **محاسبه مجموع کل Fitness جمعیت:**

$$\sum_{i=1}^{4} F(x_i) = 36 + 37 + 28 + 1 = 102$$

2. **محاسبه احتمال انتخاب برای هر کروموزوم:**

احتمال انتخاب کروموزوم x_i برابر است با نسبت Fitness آن به مجموع کل Fitness.

$$p_i = \frac{F(x_i)}{\sum F(x_j)}$$

- $p_1 = \frac{36}{102} \approx 0.353 = 35.3\ \%$
- $p_2 = \frac{37}{102} \approx 0.363 = 36.3\ \%$
- $p_3 = \frac{28}{102} \approx 0.275 = 27.5\ \%$
- $p_4 = \frac{1}{102} \approx 0.009 = 0.9\ \%$

د)

اگر بخواهیم با این احتمالات، ۲ والد برای تولید نسل بعد انتخاب کنیم، کدام کروموزوم‌ها شانس بیشتری دارند؟ چرا؟

برای انتخاب **دو والد** با روش چرخ رولت، باید فرآیند انتخاب (چرخاندن رولت) را **دو بار به صورت مستقل** انجام دهیم. دو کروموزومی که **بیشترین شانس** را برای انتخاب شدن به عنوان این دو والد دارند، به ترتیب $x_2 = 2$ و $x_1 = 1$ هستند.

(نکته: چون چرخ رولت یک روش احتمالی است، انتخاب این دو قطعی نیست و صرفاً بالاترین احتمال را دارند. حتی ممکن است یک کروموزوم با شانس بالا، دو بار انتخاب شود).

سوال دوم: عملگر (Crossover) ترکیب

۲ سوال دوم: عملگر Crossover (ترکیب)

دو کروموزوم باینری والد به صورت زیر داریم:

$$P_1 = 11010110$$

$$P_2 = 01101101$$

الف)

عملگر One-Point Crossover را با نقطه برش در موقعیت ۳ (بعد از بیت سوم) اعمال کنید و دو فرزند حاصل (C_1 و C_2) را محاسبه کنید.

نقطه برش بعد از بیت سوم قرار می‌گیرد. کروموزوم‌ها به دو بخش تقسیم شده و بخش‌های دوم آن‌ها با هم جابجا می‌شوند.

- $P_1 = 110 \mid 10110$
- $P_2 = 011 \mid 01101$

فرزندان حاصل:

- $C_1 = 110 \mid 01101 \implies C_1 = 11001101$
- $C_2 = 011 \mid 10110 \implies C_2 = 01110110$

ب)

حال همین عملگر را با نقطه برش در موقعیت ۵ (بعد از بیت پنجم) اعمال کنید و فرزندان جدید (C_1' و C_2') را به دست آورید.

نقطه برش بعد از بیت پنجم قرار می‌گیرد.

- $P_1 = 11010 \mid 110$
- $P_2 = 01101 \mid 101$

فرزندان جدید حاصل:

- $C_1' = 11010 \mid 101 \implies C_1' = 11010101$
- $C_2' = 01101 \mid 110 \implies C_2' = 01101110$

ج)

فرض کنید این کروموزوم‌های باینری نمایش اعداد صحیح در بازه $[0, 255]$ هستند. مقدار اعشاری هر یک از والدین و فرزندان (در هر دو حالت) را محاسبه کنید.

فرض شده کروموزوم‌ها نمایش باینری اعداد صحیح در بازه $[0, 255]$ هستند (اعداد ۸ بیتی بدون علامت).

والدین:

- $P_1 =$

  $11010110_2 =$

  $1 \cdot 2^7 + 1 \cdot 2^6 + 0 \cdot 2^5 + 1 \cdot 2^4 + 0 \cdot 2^3 + 1 \cdot 2^2 + 1 \cdot 2^1 + 0 \cdot 2^0 =$

  $128 + 64 + 16 + 4 + 2 = 214$

- $P_2 =$

  $01101101_2 =$

  $0 \cdot 2^7 + 1 \cdot 2^6 + 1 \cdot 2^5 + 0 \cdot 2^4 + 1 \cdot 2^3 + 1 \cdot 2^2 + 0 \cdot 2^1 + 1 \cdot 2^0 =$

  $64 + 32 + 8 + 4 + 1 = 109$

فرزندان (برش در نقطه ۳):

- $C_1 = 11001101_2 = 128 + 64 + 8 + 4 + 1 = 205$
- $C_2 = 01110110_2 = 64 + 32 + 16 + 4 + 2 = 118$

فرزندان (برش در نقطه ۵):

- $C_1' = 11010101_2 = 128 + 64 + 16 + 4 + 1 = 213$
- $C_2' = 01101110_2 = 64 + 32 + 8 + 4 + 2 = 110$

د)

با مقایسه نتایج دو نقطه برش، توضیح دهید که تغییر موقعیت نقطه برش چه تأثیری بر تنوع ژنتیکی فرزندان دارد؟

تغییر نقطه برش، نحوه ترکیب اطلاعات ژنتیکی والدین را تغییر می‌دهد و منجر به تولید فرزندان کاملاً متفاوتی می‌شود.

- **حفظ بلوک‌های ساختمانی (Building Blocks)**: نقطه برش تعیین می‌کند که کدام بخش از ژن‌های والدین دست‌نخورده به فرزندان منتقل شود.
- وقتی نقطه برش به **ابتدا یا انتهای کروموزوم** نزدیک باشد (مثلاً ۱ یا ۷)، فرزندان حاصل شباهت بسیار زیادی به یکی از والدین خواهند داشت و تغییرات جزئی خواهند بود. این کار تنوع را کاهش می‌دهد.
- وقتی نقطه برش در **وسط کروموزوم** قرار می‌گیرد (مثلاً ۳، ۴ یا ۵)، ترکیب بزرگتری از ژن‌ها صورت می‌گیرد. این امر باعث می‌شود فرزندان ویژگی‌های هر دو والد را به میزان بیشتری به ارث ببرند و افرادی کاملاً جدید ایجاد شوند که می‌تواند تنوع ژنتیکی جمعیت را افزایش دهد.
- **نتیجه‌گیری:** در این مثال، هر دو نقطه برش فرزندانی تولید کردند که مقادیر اعشاری آن‌ها بین مقادیر والدین (109 و 214) قرار دارد. اما ترکیب ژن‌ها متفاوت است. انتخاب محل نقطه برش یکی از پارامترهای مهم در الگوریتم ژنتیک است که بر سرعت همگرایی و میزان کاوش (Exploration) در فضای جستجو تأثیر مستقیم دارد. نقطه برش مرکزی‌تر معمولاً به کاوش بیشتر و تنوع بالاتر منجر می‌شود.

سوال سوم: عملگر (Mutation) جهش

۳ سوال سوم: عملگر Mutation (جهش)

یک کروموزوم باینری با طول ۱۰ بیت داریم:

$$C = 1011001011$$

نرخ جهش (Mutation Rate) برابر با $p_m = 0.1$ است.

الف)

احتمال اینکه هیچ بیتی از این کروموزوم جهش نیابد چقدر است؟

جهش هر بیت یک رویداد مستقل است.

- احتمال جهش یک بیت: $p_m = 0.1$
- احتمال جهش نیافتن یک بیت: $1 - p_m = 1 - 0.1 = 0.9$

برای اینکه هیچ‌کدام از ۱۰ بیت جهش نکنند، باید احتمال جهش نیافتن برای هر ۱۰ بیت ضرب متوالی شود:

$$P(\textit{without miutation}) = (1 - p_m)^{10} = (0.9)^{10} \approx 0.3487$$

بنابراین، تقریباً ٪۳۴.۸۷ احتمال دارد که این کروموزوم پس از عملگر جهش، هیچ تغییری نکند.

ب)

احتمال اینکه دقیقاً یک بیت جهش یابد چقدر است؟

این یک مسئله توزیع دوجمله‌ای است با:

- تعداد آزمایش‌ها (طول کروموزوم): $n = 10$
- تعداد موفقیت‌ها (تعداد جهش‌ها): $k = 1$

- احتمال موفقیت (نرخ جهش): $p = p_m = 0.1$

فرمول احتمال دوجمله‌ای: $P(X = k) = \binom{n}{k} p^k (1-p)^{n-k}$

$$P(X = 1) = \binom{10}{1} (0.1)^1 (0.9)^{10-1} = 10 \times 0.1 \times (0.9)^9 = 1 \times (0.9)^9 \approx 0.3874$$

بنابراین، تقریباً ۳۸.۷۴٪ احتمال دارد که دقیقاً یک بیت از این کروموزوم جهش یابد.

ج)

فرض کنید عملگر Bit-Flip Mutation روی این کروموزوم اعمال شده و بیت‌های موقعیت ۲ و ۷ جهش یافته‌اند. کروموزوم جدید (C') را بنویسید.

در این عملگر، بیت مورد نظر معکوس می‌شود (۰ به ۱ و ۱ به ۰).

- کروموزوم اصلی: $C = 1\underline{0}11001\underline{1}011$
- بیت در موقعیت ۲ (از چپ) 0 است. پس از جهش 1 می‌شود.
- بیت در موقعیت ۷ (از چپ) 1 است. پس از جهش 0 می‌شود.

کروموزوم جدید (C') به صورت زیر خواهد بود:

$$C' = 1\underline{1}11000\underline{0}011$$

د)

اگر این کروموزوم نمایش یک عدد صحیح بدون علامت باشد، مقدار اعشاری کروموزوم قبل و بعد از جهش را محاسبه کرده و میزان تغییر را بیان کنید.

مقدار اعشاری قبل از جهش:

- $C = 1011001011_2 = 2^9 + 2^7 + 2^6 + 2^3 + 2^1 + 2^0 = 512 + 128 + 64 + 8 + 2 + 1 = 715$

مقدار اعشاری بعد از جهش:

- $C' = 1111000011_2 = 2^9 + 2^8 + 2^7 + 2^6 + 2^1 + 2^0 = 512 + 256 + 128 + 64 + 2 + 1 = 963$

میزان تغییر:

$$963 - 715 = 248$$

جهش باعث افزایش مقدار اعشاری کروموزوم به اندازه ۲۴۸ واحد شد.

ه)

آیا افزایش نرخ جهش به $p_m = 0.3$ می‌تواند مفید باشد؟ در چه شرایطی؟

نرخ جهش 0.3 (یعنی ۳۰٪) برای یک الگوریتم ژنتیک استاندارد **بسیار بالا** محسوب می‌شود و معمولاً **مفید نیست**، اما شرایط خاصی وجود دارد که ممکن است به طور موقت مفید باشد.

- **مضرات نرخ جهش بالا:**

1. **تخریب راه‌حل‌های خوب:** الگوریتم ژنتیک با ترکیب (Crossover) و انتخاب (Selection) سعی می‌کند ویژگی‌های خوب را در نسل‌ها حفظ و تقویت کند. نرخ جهش بالا این فرآیند را مختل می‌کند و کروموزوم‌های شایسته را به صورت تصادفی از بین می‌برد.

2. **تبدیل شدن به جستجوی تصادفی:** با نرخ جهش بالا، الگوریتم بخش زیادی از ماهیت هوشمندانه خود را از دست می‌دهد و به یک جستجوی تصادفی (Random Search) شبیه می‌شود که کارایی بسیار پایینی دارد.

- **شرایطی که ممکن است مفید باشد:**

1. **فرار از بهینه محلی (Local Optima):** اگر جمعیت برای مدت طولانی در یک نقطه بهینه محلی گیر کرده باشد و پیشرفتی نکند (همگرایی زودرس)، افزایش موقتی نرخ جهش می‌تواند با ایجاد تغییرات بزرگ و ناگهانی، جمعیت را از آن نقطه خارج کرده و به کاوش در بخش‌های دیگری از فضای جستجو وادار کند.

2. **افزایش تنوع:** در ابتدای اجرای الگوریتم یا زمانی که تنوع ژنتیکی جمعیت به شدت کاهش یافته است، یک نرخ جهش بالاتر می‌تواند به سرعت تنوع را افزایش دهد.

نتیجه‌گیری نهایی: نرخ جهش $p_m = 0.3$ به عنوان یک پارامتر ثابت برای کل اجرای الگوریتم، تقریباً همیشه مضر است. اما استفاده از آن به عنوان یک **راهکار داینامیک و موقتی** برای مقابله با همگرایی زودرس، می‌تواند در شرایط خاصی مفید واقع شود.

پیاده‌سازی الگوریتم ژنتیک برای انتخاب ویژگی در دیتاست iris

منطق کلی این سوال این است: ما می‌خواهیم بهترین زیرمجموعه از ۴ ویژگی موجود در دیتاست Iris را پیدا کنیم. «بهترین» یعنی زیرمجموعه‌ای که وقتی با آن یک مدل طبقه‌بندی KNN را آموزش می‌دهیم، بالاترین دقت را روی داده‌های تست به ما بدهد و در عین حال، تا حد امکان کمترین تعداد ویژگی را داشته باشد.

این کار را با الگوریتم ژنتیک انجام می‌دهیم. هر «کروموزوم» در جمعیت ما، یک راه حل کاندید است (یعنی یک پیشنهاد برای انتخاب یا عدم انتخاب هر ویژگی).

معرفی الگوریتم K-Nearest Neighbors (KNN):

الگوریتم k تا نزدیک‌ترین همسایه (KNN) یکی از ساده‌ترین و در عین حال کاربردی‌ترین الگوریتم‌های یادگیری ماشین با نظارت (Supervised Learning) است که عمدتاً برای مسائل دسته‌بندی (Classification) استفاده می‌شود. این الگوریتم از نوع «یادگیری تنبل» (Lazy Learning) است؛ به این معنی که در مرحله آموزش مدل ریاضی پیچیده‌ای نمی‌سازد، بلکه تمام داده‌های آموزشی را در حافظه ذخیره می‌کند.

نحوه عملکرد:

1. **محاسبه فاصله:** برای پیش‌بینی کلاس یک داده جدید، الگوریتم فاصله آن داده را با تمام داده‌های آموزشی موجود (معمولاً با استفاده از معیار "فاصله اقلیدسی") محاسبه می‌کند.
2. **انتخاب همسایه‌ها:** سپس k داده‌ای که کمترین فاصله را با داده جدید دارند (نزدیک‌ترین همسایه‌ها) انتخاب می‌کند. مقدار k یک عدد صحیح است که توسط کاربر تعیین می‌شود (مثلاً در این پروژه $k = 3$ فرض شده است).
3. **تصمیم‌گیری:** در نهایت، کلاس داده جدید بر اساس «رای‌گیری اکثریت» (Majority Voting) مشخص می‌شود؛ یعنی هر کلاسی که در میان این k همسایه بیشترین نماینده را داشته باشد، به عنوان کلاس داده جدید در نظر گرفته می‌شود.

نقش KNN در این پروژه:

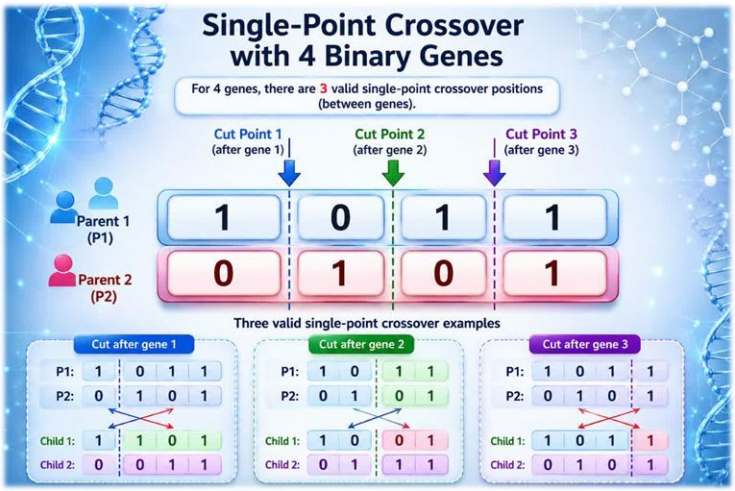
در این پیاده‌سازی، از الگوریتم KNN درون تابع برازش (Fitness Function) استفاده شده است. الگوریتم ژنتیک زیرمجموعه‌های مختلفی از ویژگی‌ها را پیشنهاد می‌دهد و KNN با استفاده از تنها همان ویژگی‌ها، داده‌ها را دسته‌بندی می‌کند. هرچه دقت دسته‌بندی KNN بالاتر باشد، امتیاز (Fitness) آن زیرمجموعه از ویژگی‌ها بیشتر خواهد بود.

داده‌ها را به صورت ٪۷۰ آموزش و ٪۳۰ تست تقسیم کنید.

در جدول، ۵۰ نمونه گل Iris-setosa و ۵۰ نمونه گل Iris-versicolor و در آخر ۵۰ نمونه گل Iris-virginica به ترتیب موجود است؛ به زنبق setosa اندیس ۱ و به زنبق versicolor اندیس ۲ و در نهایت به زنبق virginica اندیس ۲ می‌دهیم. طبق صورت سوال، ۳۵ داده اول هر دسته نمونه را داده های آموزشی و ۱۵ داده مابقی هر دسته نمونه را داده های تست در نظر می‌گیریم.

در ضمن، این الگوریتم KNN با K=3 اگر ۳ همسایه نزدیک‌اش اندیس‌های ۱ و ۲ و ۳ (یعنی هر سه متفاوت) بودند، کوچکترین اندیس یعنی ۱ را انتخاب می‌کند. برای Elitism و بازگشت بهترین کروموزوم‌ها (نخبه‌گرایی) ۲ تا از بهترین شبکه‌ها (۱۰ درصد کل شبکه‌های یک نسل) را همیشه بدون تغییر و جهش به نسل بعد منتقل می‌کنیم و ۱۸ تا شبکه‌های باقی‌مانده را از Crossover به دست می‌آوریم؛ یعنی از ۱۸ والد که از تورنومنت جان سالم به در برده‌اند.

بنا به شکل زیر هم برای Crossover کردن دو کروموزوم ۴ ژنی، نیاز به یک محل برش با فضای حالت ۳ تایی داریم که باید رندوم انتخاب شود:



تنظیمات اولیه:

Elementry AI HW2 (Iris Question)

Clearing Command window and Workspace:

```
close all;
clear;
clc;
```

Fixing Random Number Generator (RNG):

{42 is just a contracted seed between programmers!}

```
rng(42);
```

تعریف هایپر پارامتر ها: (چون تعداد نخبه‌های نسل بعد باید زوج باشد، من به جای گرفتن تعداد Elitism و بررسی اینکه زوج است یا فرد، همان اول نصف Elitism را می‌گیرم: (تعداد نخبه هایی که بدون تغییر به نسل بعد می‌روند.)

Declaring Hyperparameters:

(Length of every chromosome is 4)

```
half_Elitism = 0;
population = 20;
generations = 50;
mutation_rate = 0.1;
crossover_rate = 0.8;
tournament_amount = 3;
% Constants:
K = 3;
trains_amount = 105;   % (150 * 0.7)
tests_amount = 45;     % (150 * 0.3)
```

ساخت یک حافظه برای کشیدن نمودار Best Fitness و Mean Fitness :

History for plotting:

```
best_fitness_history = zeros(1, generations);
mean_fitness_history = zeros(1, generations);
```

لود کردن فایل CSV و تبدیل کلمات ستون آخر به اعداد: (همانطور که بحث شد.)

Loading CSV File:

```
One_Block = true;
Elitism = 2 * half_Elitism;
table_datas = readtable("iris.csv"); % table (150 * 5)
x = table2array(table_datas(:, 1:4));
y = zeros(150, 1);

for i = 1:150
    if strcmp(table_datas{i,5} , 'Iris-setosa')
        y(i) = 1;
    elseif strcmp(table_datas{i,5} , 'Iris-versicolor')
        y(i) = 2;
    elseif strcmp(table_datas{i,5} , 'Iris-virginica')
        y(i) = 3;
    end
end
```

نرمال سازی داده های ورودی (طول و عرض گلبرگ و کاسبرگ):

(تابع Random_Chromosome() همراه تعریفش جلوتر بیان می‌شود.) {رفتن به توضیحات توابع}

Normalizing x with this formula:

$$x \longleftarrow \frac{x - \mu_x}{\sigma_x}$$

```
X_mean = mean(x, 1); % axis = 1
X_std = std(x, 0, 1); % flag = 0 , axis = 1
% update:
x = (x - X_mean) ./ X_std;
creatures = zeros(population, 4);
fitnesses = zeros(population, 1);

% initialization
for i = 1:population
    creatures(i,:) = Random_Chromosome();
end
```

داده‌های تست و آموزش را با همان شیوه که گفته شد جدا می‌کنیم:

Separating both "Train" and "Test" parts:

```
Trains = zeros(trains_amount, 4);      % size = (105 , 4)
Tests = zeros(tests_amount, 4);        % size = (45  , 4)
Y_Trains = zeros(trains_amount, 1);    % size = (105 , 1)
Y_Tests = zeros(tests_amount, 1);      % size = (45  , 1)

for i = 0:2
    Trains(35*i+1 : 35*i+35 , :) = x(50*i+1 : 50*i+35 , :);
    Tests(15*i+1 : 15*i+15, :) = x(50*i+36 : 50*i+50 , :);
    Y_Trains(35*i+1 : 35*i+35 , 1) = y(50*i+1 : 50*i+35 , 1);
    Y_Tests(15*i+1 : 15*i+15, 1) = y(50*i+36 : 50*i+50 , 1);
end
```

حلقه اصلی: (در آخر حلقه داده‌ها ذخیره شده‌اند.)

Main Loop:

```
for Gen = 1:generations
    % Copying
    chromosomes = creatures;
    % Evaluating all of the Fitnesses:
    fitnesses = Fitness(creatures, K, Trains, Y_Trains, Tests, Y_Tests);
    [fitnesses, idx] = sort(fitnesses, 'descend');
    % Elitism (2 Networks):
    chromosomes(1,:) = creatures(idx(1),:);
    chromosomes(2,:) = creatures(idx(2),:);
    % Performing Selections, Crossovers and Mutations (18 Networks):
    chromosomes = Selection_Crossover(chromosomes, idx, crossover_rate, tournament_amount, Elitism);
    chromosomes = MUTATION(chromosomes, mutation_rate, Elitism);
    % Validation and Correction
    chromosomes = Validate(chromosomes);
    % Pasting the changes
    creatures = chromosomes;
    % Saving history
    best_fitness_history(Gen) = max(fitnesses);
    mean_fitness_history(Gen) = mean(fitnesses);
end
```

کد plotting:

Plotting:

```
figure;
G = 1:generations;
plot(G, best_fitness_history, '-b', 'LineWidth', 2);
hold on;
plot(G, mean_fitness_history, '-r', 'LineWidth', 2);
title('Convergence Graph');
xlabel('Generations');
ylabel('Fitness');
legend('Best Fitness', 'Mean Fitness');
xlim([1,generations]);
ylim([-10,110]);
axis manual;
grid on;
% Declaring Legend for better demonstrations:
legend("Position", [0.67 , 0.13 , 0.22 , 0.08]);
```

توضیحات توابع:

۱. تابع Random_Chromosome (تولید کروموزوم تصادفی)

- **نحوه کارکرد:** این تابع یک آرایه باینری (شامل 0و 1) با ابعاد 4×1 به صورت کاملاً تصادفی تولید می‌کند.
- **دلیل استفاده:** الگوریتم ژنتیک برای شروع به یک جمعیت اولیه نیاز دارد. هر کروموزوم نشان‌دهنده حضور (1) یا عدم حضور (0) یک ویژگی از دیتاست است. این تابع تضمین می‌کند که کروموزومی با مقدار تمام صفر ($[0, 0, 0, 0]$) تولید نشود تا حداقل یک ویژگی برای ارزیابی مدل وجود داشته باشد.
- **معماری:**

Functions:

Function (1):

```
94  function out = Random_Chromosome()
95      out = randi([0 1], 1, 4);
96      while all(out == 0)
97          out = randi([0 1], 1, 4);
98      end
99  end
```

۲. تابع Distance (محاسبه فاصله وزن‌دار)

- **نحوه کارکرد:** این تابع فاصله اقلیدسی میان یک داده جدید و داده‌های آموزشی را محاسبه می‌کند، اما با این تفاوت که ژن‌های کروموزوم به عنوان وزن در فرمول دخالت داده می‌شوند:

$$dis = \sqrt{\sum(Chr \cdot (M1 - M2)^2)}$$

- **دلیل استفاده:** برای اینکه الگوریتم KNNتنها بر اساس ویژگی‌های انتخاب‌شده توسط کروموزوم فعلی فواصل را بسنجد، ویژگی‌هایی که ژن آن‌ها 0 است، در محاسبه فاصله خنثی شده و تاثیری در انتخاب همسایه‌ها نخواهند داشت.
- **معماری:**

Function (2):

```
100  function dis = Distance(Chr, M1, M2)
101      dis = sqrt( sum(Chr .* (M1 - M2).^2) );
102  end
```

۳. تابع KNN (الگوریتم نزدیک‌ترین همسایه‌ها)

- **نحوه کارکرد:** با دریافت کروموزوم فعلی، تعداد همسایه‌ها (k)، داده‌های آموزش و یک داده جدید، فواصل را توسط تابع Distance محاسبه کرده و آن‌ها را مرتب می‌کند. سپس kرکورد که کمترین فاصله را دارند انتخاب کرده و برچسبی که بیشترین تکرار (Mode) را بین آن‌ها دارد، به عنوان خروجی پیش‌بینی می‌کند.
- **دلیل استفاده:** این تابع به عنوان ارزیاب (Classifier) اصلی سیستم عمل می‌کند تا مشخص شود ترکیب ویژگی‌های پیشنهادی کروموزوم، تا چه حد قادر به دسته‌بندی صحیح داده‌های جدید است.
- **معماری:**

Function (3):

```
103  function predict = KNN(Chr, k, datas, labels, new_data)
104      sz = size(datas, 1);
105      DISTANCES = zeros(sz,1);
106      for i = 1:sz
107          DISTANCES(i) = Distance(Chr, datas(i,:), new_data);
108      end
109      [~ , idx] = sort(DISTANCES);
110      lb = zeros(k,1);
111      for i = 1:k
112          N = idx(i);
113          lb(i) = labels(N);
114      end
115      predict = mode(lb);
116  end
```

۴. تابع Fitness (تابع برازش / ارزیابی عملکرد)

- **نحوه کارکرد:** این تابع کل جمعیت کروموزوم‌ها را دریافت کرده و برای تک‌تک آن‌ها، الگوریتم KNN را روی تمام داده‌های تست اجرا می‌کند. در نهایت، تعداد پیش‌بینی‌های صحیح (یا درصد دقت) هر کروموزوم را به عنوان امتیاز برازش آن برمی‌گرداند.

- **دلیل استفاده:** قلب الگوریتم ژنتیک تابع برازش است؛ این تابع مشخص می‌کند که کدام کروموزوم‌ها (ترکیب ویژگی‌ها) عملکرد بهتری داشته‌اند تا در فرآیند انتخاب برای تولید نسل بعد، شانس بیشتری داشته باشند.
- **معماری:**

Function (4):

```
117  function F = Fitness(Chrs, k, datas, labels, Tests, Tests_labels)
118      T_amount = size(Tests, 1);
119      pop = size(Chrs, 1);
120      F = zeros(pop, 1);
121
122      for p = 1:pop
123          for t = 1:T_amount
124              tst = Tests(t, :);
125              answer = Tests_labels(t);
126              prd = KNN(Chrs(p,:), k, datas, labels, tst);
127
128              if prd == answer
129                  F(p) = F(p) + 1;
130              end
131          end
132          % Converting to Percent
133          F(p) = (F(p) / T_amount) * 100;
134      end
135  end
136
```

۵. تابع Selection_Crossover (انتخاب والدین و تقاطع)

- **نحوه کارکرد:** این تابع از روش «انتخاب تورنمنت» (Tournament Selection) استفاده می‌کند. چند کروموزوم به صورت تصادفی انتخاب شده و بهترینِ آن‌ها به عنوان والدین برگزیده می‌شوند. سپس با در نظر گرفتن احتمال تقاطع (CROSS_RATE)، یا عمل ترکیب روی آن‌ها انجام می‌شود و یا عیناً به نسل بعد منتقل می‌شوند. این تابع کروموزوم‌های نخبه (Elites) را دستکاری نمی‌کند.
- **دلیل استفاده:** شبیه‌سازی قانون بقای اصلح در طبیعت. این تابع باعث می‌شود کروموزوم‌های قوی‌تر شانس بیشتری برای انتقال ژن‌های خوب خود به نسل بعدی داشته باشند.
- **معماری:**

Function (5):

```
137  function RES = MUTATION(Chrs, Rate, Elite)
138      pop = size(Chrs, 1);
139      RES = Chrs;
140      MUT = rand(pop - Elite, 4);
141      binary_MUT = (MUT >= (1 - Rate));
142      RES(Elite+1:pop , :) = xor(RES(Elite+1:pop , :) , binary_MUT);
143  end
```

۶. تابع Cross_Over (عملگر تقاطع / Crossover)

- **نحوه کارکرد:** این تابع عمل تقاطع تک‌نقطه‌ای (Single-point Crossover) را روی دو کروموزوم والد انجام می‌دهد. یک نقطه برش تصادفی (مثلاً بین ژن‌های 1تا 3) انتخاب شده و بخش انتهایی دو والد با یکدیگر جابجا می‌شوند تا دو فرزند جدید تولید شوند.
- **دلیل استفاده:** برای تبادل اطلاعات ژنتیکی بین کروموزوم‌های موفق و ایجاد ترکیب‌های جدیدی از ویژگی‌ها که ممکن است به دقت بالاتری نسبت به والدین خود برسند.
- **معماری:**

Function (6):

```
144  function out = Validate(Chrs)
145      pop = size(Chrs, 1);
146      for i = 1:pop
147          if all(Chrs(i,:)==0)
148              r = randi([1,4]);
149              Chrs(i,r) = 1;
150          end
151      end
152      out = Chrs;
153  end
```

۷. تابع MUTATION (عملگر جهش)

- **نحوه کارکرد:** این تابع یک ماسک باینری تصادفی بر اساس نرخ جهش (Mutation_Rate) می‌سازد و با استفاده از عملگر منطقی XOR، مقدار برخی از ژن‌ها (بیت‌ها) را در جمعیت جدید معکوس می‌کند (0 به 1و برعکس).

- **دلیل استفاده:** جهش باعث حفظ تنوع ژنتیکی در جمعیت می‌شود و از گیر افتادن الگوریتم در بهینه‌های محلی (Local Optima) جلوگیری می‌کند. این کار به الگوریتم اجازه می‌دهد فضاهای جدیدی از مسئله را جستجو کند.
- **معماری:**

Function (7):

```
154  function [C1 , C2] = Cross_Over(P1, P2)
155      cut_pose = randi([1,3]);
156      C1 = zeros(1,4);
157      C1(1:cut_pose) = P1(1:cut_pose);
158      C1(cut_pose+1:end) = P2(cut_pose+1:end);
159      C2 = zeros(1,4);
160      C2(1:cut_pose) = P2(1:cut_pose);
161      C2(cut_pose+1:end) = P1(cut_pose+1:end);
162  end
```

۸. تابع Validate (اعتبارسنجی کروموزوم‌ها)

- **نحوه کارکرد:** پس از انجام تقاطع و جهش، این تابع کل جمعیت را بررسی می‌کند تا مطمئن شود کروموزوم نامعتبری (مثلاً آرایه‌ای که همه ژن‌های آن 0شده باشد) به وجود نیامده باشد. در صورت یافتن چنین حالتی، یک بیت تصادفی را به 1تغییر می‌دهد.
- **دلیل استفاده:** جلوگیری از بروز خطای منطقی در محاسبه فاصله و اجرای KNN؛ زیرا اگر کروموزوم تماماً 0باشد، هیچ ویژگی برای محاسبه وجود نخواهد داشت و مدل با خطا (یا رفتار غیرمنطقی) مواجه می‌شود.
- **معماری:**

Function (8):

```
function out = Selection_Crossover(Chrs, index, CROSS_RATE, TOUR_AMNT, ELITE)
    pop = size(Chrs, 1);
    out = Chrs;
    counter = 3;
    c1 = zeros(1,4);
    c2 = zeros(1,4);
    repeating = (pop - ELITE) / 2;
    for i = 1:repeating
        attenders = randi([1,pop],TOUR_AMNT,1);
        attenders = sort(attenders, 'ascend');
        % Winners: attenders(1) & attenders(2)
        idx1 = index(attenders(1));
        idx2 = index(attenders(2));
        p1 = Chrs(idx1,:);
        p2 = Chrs(idx2,:);
        % Do we need Crossover or not?
        need = rand;
        if need < CROSS_RATE
            [c1 , c2] = Cross_Over(p1,p2);
        else
            c1 = p1;
            c2 = p2;
        end
        out(counter,:) = c1;
        counter = counter + 1;
        out(counter,:) = c2;
        counter = counter + 1;
    end
end
```